

Documentation technique : Installation de Mistral AI local

Table des matières

1) Introduction.....	1
1.1) Définition.....	1
1.2) Objectif.....	1
2) Installations et configurations.....	1
2.1) Installation des dépendances.....	1
2.2) Installation et configuration de llama.cpp.....	1
2.3) Installation et utilisation de Mistral AI en local.....	2
3) Utilisation de Mistral AI en mode graphique.....	2
3.1) Installation de Ollama et de Mistral.....	2
3.2) Installation et utilisation du support graphique.....	3
4) Utilisation de MistralAI avec OpenWebUI (Alternatif).....	3
4.1) Installation de Docker.....	3
4.2) Installation et configuration d'Ollama et de Mistral.....	4
4.3) Déploiement d'OpenWebUI et configurations.....	5

1) Introduction

1.1) Définition

Une intelligence artificielle locale est une intelligence artificielle qui ne nécessite pas de connexion à Internet pour pouvoir être utilisée, elle est stockée sur un disque et s'utilise à travers un logiciel de lignes de commandes.

1.2) Objectif

L'objectif de cette documentation est de montrer les étapes permettant d'installer l'intelligence artificielle **Mistral AI** en local sur une machine virtuelle Debian Linux. On devra l'utiliser en mode commande avec Llama et en graphique avec ChatboxAI.

2) Installations et configurations

2.1) Installation des dépendances

Pour pouvoir effectuer chacune des commandes sans avoir d'erreurs, il faut installer deux outils :

- **curl** (pour les installations) : **sudo apt install curl -y**
- **cmake** (pour la compilation du dossier llama.cpp) : **sudo apt install cmake -y**

2.2) Installation et configuration de llama.cpp

Pour pouvoir utiliser notre IA en local, il faut installer l'outil llama.cpp qui va permettre d'héberger et d'utiliser notre IA.

Pour installer le dépôt llama.cpp, il faut utiliser la commande :

git clone <https://github.com/ggerganov/llama.cpp>

Ensuite nous devons nous placer dans le dossier avec **cd llama.cpp**

Dans le dossier de llama.cpp nous devons créer un sous dossier **build**. Le sous dossier build se construit avec la commande : **mkdir build**. Après s'être placé dans build on doit utiliser la fameuse commande **cmake** avec **cmake ..** pour configurer le dossier.

Enfin pour le compiler il faut utiliser cette commande :

cmake --build . --config Release -j

2.3) Installation et utilisation de Mistral AI en local

Pour utiliser Mistral, on doit d'abord le télécharger dans notre dossier build, on doit donc se placer dans ce dossier et utiliser la commande suivante :

curl -L -o mistral-7b-instruct-v0.3.Q4_K_M.gguf
https://huggingface.co/MaziyarPanahi/Mistral-7B-Instruct-v0.3-GGUF/resolve/main/Mistral-7B-Instruct-v0.3.Q4_K_M.gguf

Pour finir, on peut utiliser le modèle avec la commande :

./bin/llama-cli -m mistral-7b-instruct-v0.3.Q4_K_M.gguf -cnv

3) Utilisation de Mistral AI en mode graphique

3.1) Installation de Ollama et de Mistral

Pour cette partie j'ai du changer de système car il n'y avait pas assez de mémoire pour la VM Debian Linux. On installera donc ollama sur Windows 11.

Pour lancer Mistral en mode graphique cette fois ci, on va utiliser une version plus complète de llama.cpp qui représente à peu près le moteur principal avec Ollama, un outil plus complet que llama.cpp qui nous permettra de faciliter l'installation de Mistral AI avec des commandes plus simples et qui nous permet par la suite d'utiliser ChatboxAI qui nous servira à faire l'affichage de Mistral AI en mode graphique.

Pour installer Ollama il faut se rendre sur le site et installer la version windows :

<https://ollama.com/download/windows>

Ensuite il faut installer Mistral sur Windows avec la commande :

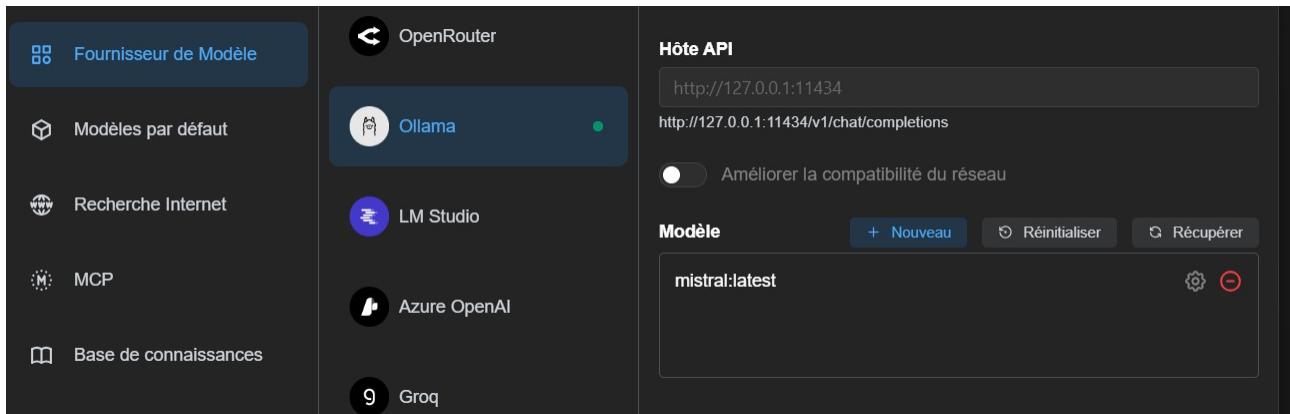
ollama pull mistral

3.2) Installation et utilisation du support graphique

Pour installer chatboxAI il faut aller sur leur site et télécharger l'installateur :

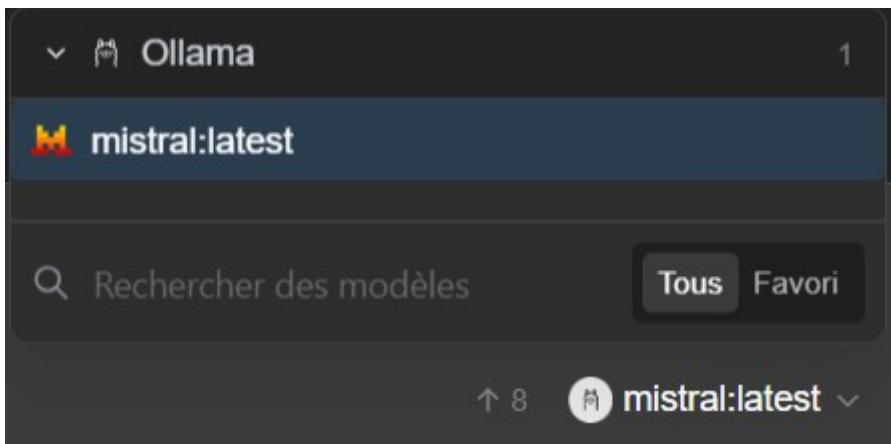
<https://chatboxai.app/fr/install?download=win64>

Lorsque le logiciel est installé, il faut ouvrir le fournisseur de modèle dans les paramètres du logiciel comme ceci :

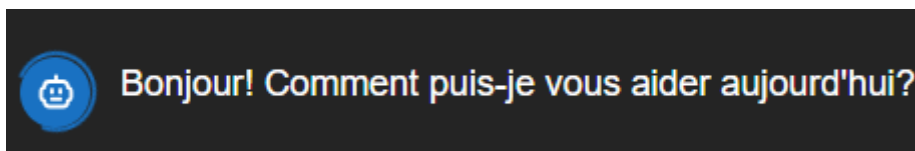


Pour ajouter un modèle, il suffit sur cette page de cliquer sur le bouton récupérer, de sélectionner le modèle dans la liste déroulante et de cliquer sur le +.

Ensuite on revient au menu principal, et on sélectionne notre modèle en bas à droite :



Résultat :



4) Utilisation de MistralAI avec OpenWebUI (Alternatif)

4.1) Installation de Docker

Pour utiliser MistralAI de façon graphique sur Debian Linux, il existe au final un moyen de le faire avec Docker.

Il faut donc tout d'abord mettre à jour les paquets systèmes avec la commande :

```
sudo apt update && sudo apt upgrade -y
```

Ensuite il faut installer les prérequis pour docker (certificats) :

```
sudo apt install -y curl apt-transport-https ca-certificates gnupg lsb-release
```

après ça on doit ajouter la clé GPG officielle de Docker :

```
sudo mkdir -p /etc/apt/keyrings curl -fsSL https://download.docker.com/linux/debian/gpg |  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Ensuite on doit ajouter le dépôt Docker aux sources de Debian :

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/debian $(lsb_release -cs) stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

Pour enfin pouvoir installer Docker avec les commandes :

```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

4.2) Installation et configuration d'Ollama et de Mistral

Pour installer Ollama il suffit d'utiliser la commande :

```
curl -fsSL https://ollama.com/install.sh | sh
```

Lorsqu'il est installé il va falloir modifier le fichier de configuration pour que OpenWebUI puisse reconnaître les modèles d'Ollama.

On doit donc ouvrir l'éditeur de configuration d'Ollama avec :

```
sudo systemctl edit ollama.service
```

Il faut se placer en dessous de la ligne `### Anything between here ...` et écrire :

```
[Service]
```

```
Environment="OLLAMA_HOST=0.0.0.0"
```

Ensuite **on sauvegarde avec Ctrl+O, Entrée, Ctrl+X**

Enfin, on doit forcer le rechargement de la configuration en redémarrant le service en utilisant les commandes :

sudo systemctl daemon-reload

sudo systemctl restart ollama

Pour finir on doit installer mistral avec

ollama pull mistral

4.3) Déploiement d'OpenWebUI et configurations

Pour déployer OpenWebUI, il va falloir utiliser la commande :

sudo docker run -d -p 3000:8080 -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main

Lorsque c'est fait on doit ouvrir Firefox, puis écrire dans la barre de recherche

<http://localhost:3000>.

On doit par la suite créer un compte administrateur, aller dans les paramètres, aller dans la section Connexions et chercher le champ d'Ollama API.

Dans le terminal Linux il va falloir chercher l'IP d'Ollama car par défaut OpenWebUI utilise une URL réservé à Windows.

L'IP se trouve avec la commande **hostname -I**.

Il faut par la suite mettre l'URL suivante dans la configuration de l'API :

http://IP_TROUVE:11434

(où IP_TROUVE est l'ip trouvé avec hostname -I)

On peut ensuite vérifier si ça marche avec l'icone de rafraichissement en haut à droite du menu.

Pour finir on revient au menu principal puis on sélectionne en haut à gauche le modèle téléchargé.

On peut désormais parler avec Mistral !

Chose à savoir : Mistral ne se lancera pas si il n'y a pas assez de RAM dans le système.